

Overview

`packages/graph-view/src/utils/filters.ts` now exports three helpers:

- `filterNodes` (lines 19-56)
- `filterEdges` (lines 57-76)
- internal `getEdgeCategory` (lines 81-88).

The file adds imports for `GraphNode`, `GraphEdge`, `FilterState`, `NodeType`, `Complexity`, `EdgeCategory`, and `EDGE_CATEGORY_MAP` (lines 1-3).

Key changes

- `filterNodes` receives a pre-computed `nodeIdToLayerIds: Map<string, Set<string>>`. *Layer checks run in $O(1)$ by looking up the set of layer IDs for each node and testing membership against `filters.layerIds`.* This replaces the previous $O(N \times L \times K)$ loop that used `layer.nodeIds.includes`.
- `filterEdges` keeps edges only if both endpoints are visible (`visibleNodeIds.has`) and if the edge's category (determined by `getEdgeCategory`) is present in `filters.edgeCategories`.
- `getEdgeCategory` iterates over `EDGE_CATEGORY_MAP` to map an edge type to its category, returning `null` for unknown types.

All functions are fully typed and documented with JSDoc comments.

Impact

- **Performance:** Constant-time layer membership checks reduce export time for large graphs (#102).
- **Correctness:** Maintains "any-layer-wins" semantics; nodes belonging to any selected layer are kept.
- **Maintainability:** Centralizes filtering logic, simplifying future extensions.
- **Observability:** Functions are pure; no new runtime side-effects.

Risks & follow-ups

- `filterNodes` drops a node if `nodeIdToLayerIds` lacks an entry for its ID. Ensure the map is populated before use.
- `filterEdges` allows edges with unknown types to pass the category filter because `getEdgeCategory` returns `null`. Verify that `EDGE_CATEGORY_MAP` covers all edge types.
- Update any components that previously relied on legacy filtering logic to import and invoke these new helpers.

- Add unit tests for edge cases (nodes with no layers, edges with unknown types) to confirm deterministic behavior.